Merge K Sorted Arr

K=4

arr

0	3	4	5	11
1	1	6	7	14
2	4	5	8	13
3	6	9	10	12

Brute-force Approaches:

All elements in 1D Arr K^2

+ Sort them

[1]

Merge Sort

if n elements

nlogn

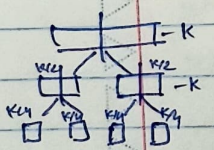
so for K^2 elements

$T.C: (K^2 \log(K^2))$

$\approx K^2 \log K$

$S.C: O(K^2) + \log(K^2)$

merge func $\approx O(K^2)$ ← recursive calls



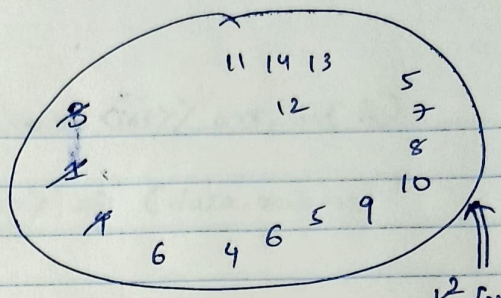
[2]

Heap Sort

$T.C: O(K^2 \log K)$

$S.C: O(1)$

[3] Using MinHeap for all elements



K^2 for insertion if step-down

or

$K^2 \log K$ (step-up).

+ $(K^2 \log(K^2))$ for deletion of K^2 elements

1	3	4	4	5	6	6	...

* No utilization of sorted property.

$T.C: O(K^2) + O(K^2 \log K)$

$\approx O(K^2 \log K)$

$S.C: O(K^2)$

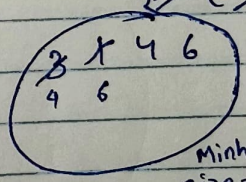
Auxiliary space.

[4] Each row is sorted in itself so,

K	3	4	5	11
	1	6	7	14
	4	5	8	13
	6	9	10	12

K

pair (data, row, col)



Minheap size = K

1	3	4	4	...

Inserting K elements into the heap:

(step-down) K or $K \log K$ (step-up)

Total elements: K^2

Remaining elements: $(K^2 - K)$

& the deletion $(K^2 - K) \log K$

So Overall,

$K \log K + K^2 \log K - K \log K$

$T.C: O(K^2 \log K)$

$S.C: O(K)$.

Approach 4: T.C: $O(K^2 \log K)$
S.C: $O(K)$.

CODE:

```
vector<int> mergeKArrays(vector<vector<int>> arr, int K)
{
  // Pair type: pair<int, pair<int, int>> ⇒ (data, row, col)

   vector<pair<int, pair<int,int>>> temp;
   for(int i=0; i<K; i++){
      temp.push_back(make_pair(arr[i][0], make_pair(i, 0)));
   }

   // Normal Minheap
   // priority_queue<int, vector<int>, greater<int>> p;
   // Our use-case:
   priority_queue<pair<int, pair<int,int>>, vector<pair<int, pair<int,int>>>,
        greater<pair<int, pair<int,int>>>> p(temp.begin(), temp.end());
```

greater closing braces

priority_queue closing braces

```
vector<int> ans; //vector which will store the sorted elements.
pair<int, pair<int,int>> Element;
int i, int j;
while(!p.empty()){
     Element = p.top();
     p.pop();
     ans.push_back(Element.first);
     i= Element.second.first;
     j= Element.second.second;
     if(j+1<K)
     p.push(make_pair(arr[i][j+1], make_pair(i,j+1)));
}

  return ans;
```

Approach: 5 : Optimized Merge Sort

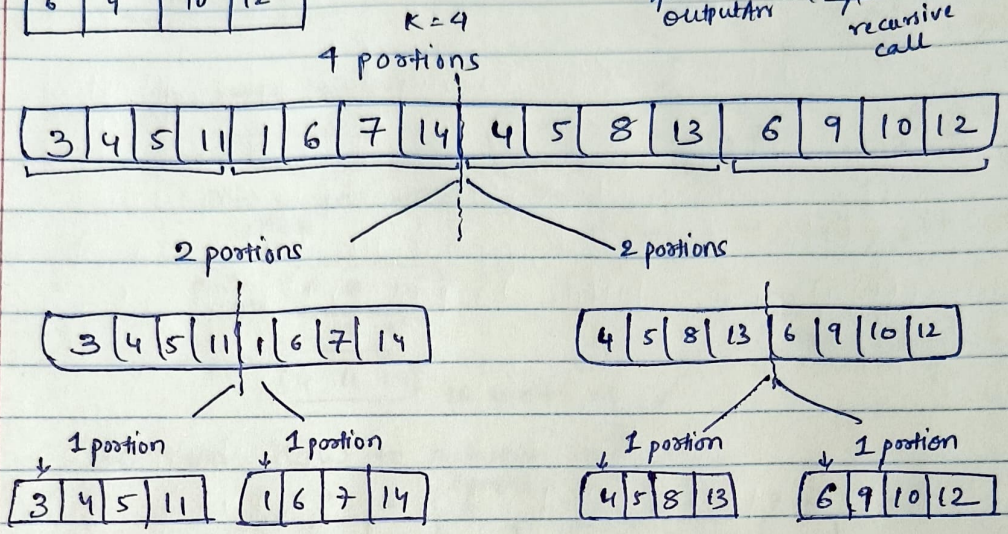
K=4

3	4	5	11
1	6	7	14
4	5	8	13
6	9	10	12

T.C: $O(N \log K)$ or $O(K \log K)$
as there are $\log K$ levels of merging, & each level processes all N elements once.

S.C: $O(N)$ + $O(\log K) + O(N)$ ← Auxiliary
↑ outputArr ↑ recursive call ↑ merge func extra array.

Ex1:

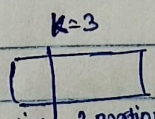


~~The idea~~

No need to further subdivide, they are sorted within themselves,
As an when (end - start == K or portion == 1) stop!.

The idea is to divide the Arr into K portions, each portion is sorted in itself, so directly, we can call merge func without each portion being further subdivided until ☒ single element.

portion == 1 stop but we need whole number, even if portion is 1.2 or 1.5



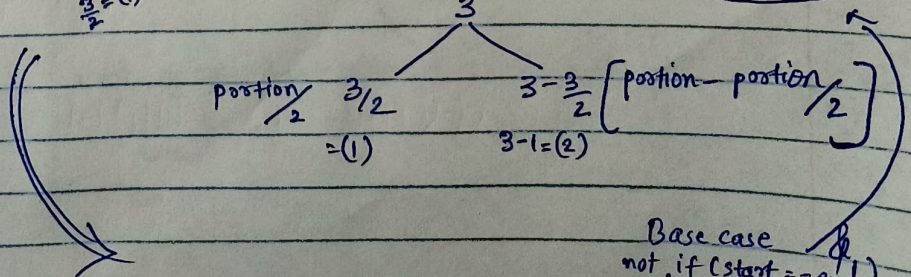
we stop so

$\frac{3}{2} = 1.5 \approx (1)$

$\frac{3}{2} = (1)$

```
if (portion < 2)
    return;
```

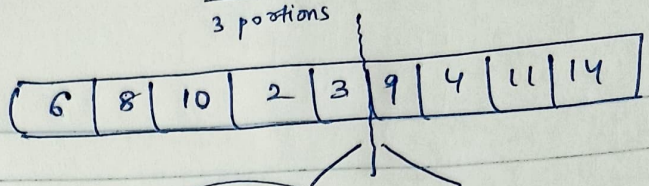
let's see & understand with the help of an example.



Ex2: K=3 portion=3

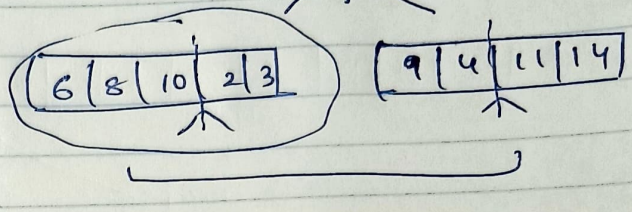
6	8	10
2	3	9
4	11	14

K=3
3 portions



We cannot divide it into half.

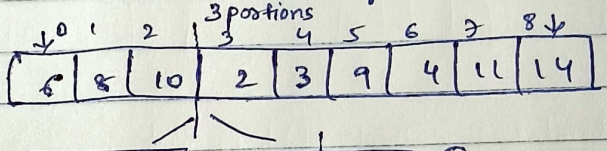
this is not sorted.



only [6 8 10] [2 3 9] [4 11 14] is sorted so

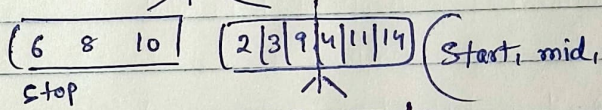
this can go wrong.

So then how to achieve it?



start = 0
end = 8
portion = 3
mid = start + (portion/2) * K - 1

no of elements in each portion



(start, mid, portion/2) (0, 2, 1)

to (mid+1, end, portion - portion/2) (3, 8, 2)

$0 + \frac{3}{2} * 3 - 1$

$0 + 1 * 3 - 1$

mid = 0 + 2 = (2)

Obviously K=3
portion = K as K lists

Another way to calculate portion =>

$\frac{end - start + 1}{K} = \frac{9}{3} = (3)$.

CODE:

```
vector<int> mergeKArrays(vector<vector<int>> arr, int K)
{
  vector<int> ans;
  for(int i=0; i<K; i++)
  for(int j=0; j<K; j++){
     ans.push_back(arr[i][j]);
  }
  mergeSort(ans, 0, ans.size()-1);
  return ans;
}
```

```
void mergeSort(vector<int>& arr, int start, int end, int K){
   if(portion<2)
   return;
   int mid = start + (portion/2) * K - 1;
   // leftside
   mergeSort(arr, start, mid, portion/2, K);
   //rightside
   mergeSort(arr, mid+1, end, portion - portion/2, K);
   merge(arr, start, mid, end);
}
```

merge func same as merge sort.